

Estimación de proyectos de software

Introducción

Conocer de antemano el tamaño aproximado de un proyecto, los recursos necesarios para llevarlo a cabo y el tiempo que tomará completarlo ofrece a los equipos de desarrollo, gerentes, sponsors e interesados una base sólida para la toma de decisiones. En la industria del software, no disponer de una estimación “adecuada” puede conducir a sobrecostos, retrasos, cumplimiento de objetivos y problemas de calidad en el producto (“t + \$ + scope + Q”). Contar con estimaciones confiables contribuye a generar confianza entre los miembros del equipo y los diferentes involucrados en un proyecto.

Existen múltiples enfoques existentes sobre las metodologías de estimación, entre las que destacan los Puntos de Caso de Uso, Puntos de Función, estimaciones ágiles como Poker Planning y Story Points, la estimación de Tres Puntos (PERT) y los modelos algorítmicos como COCOMO. Cada método mide desde una perspectiva diferente y responde a distintas necesidades y contextos de proyecto. Algunas técnicas hacen hincapié en la complejidad del software, otras en la colaboración del equipo y otras en el análisis estadístico de la incertidumbre. En todos los casos, se persigue alcanzar la mayor precisión y fiabilidad posible, teniendo en cuenta las limitaciones y particularidades de cada proyecto.

Contexto

Para enmarcar adecuadamente la importancia de la estimación, es útil partir de lo que se conoce como la Ley de Parkinson, formulada por Cyril Northcote Parkinson en 1957. De manera simplificada, Parkinson planteó que “el trabajo se expande hasta llenar el tiempo disponible para su realización”. Aunque esta ley nació de la observación sobre la burocracia en la administración pública británica, su principio se ha extrapolado al campo de la gestión de proyectos, incluida la industria del software. Parkinson observó que, incluso si la carga de trabajo objetiva disminuye, la cantidad de personal y la burocracia seguían creciendo a un ritmo constante, mostrando que las organizaciones tienden a consumir cualquier recurso que se les otorgue, sea tiempo, personal o dinero. Visto desde el ángulo de la planificación de proyectos, la Ley de Parkinson denota la necesidad de estimar de forma adecuada y sobre la tendencia de los equipos a ocupar todo el plazo establecido, independientemente de si la tarea podría realizarse en menor tiempo. Ergo, una estimación realista y cuidadosamente diseñada puede moderar los efectos de esta expansión innecesaria, ayudando a los equipos a delimitar sus esfuerzos y a emplear sus recursos de forma más eficiente.

La necesidad de estimar el tamaño de un proyecto de software radica, entonces, en la búsqueda de claridad para cumplir con requisitos, plazos y expectativas. La estimación no es una actividad aislada e independiente, sino que forma parte de un conjunto de procesos relacionados con la planificación: gestión de riesgos, distribución de recursos, definición de planes de contingencia y toma de decisiones sobre el alcance, justificación y la viabilidad

del producto a desarrollar. Un cálculo de esfuerzo que no sea “correcto” puede llevar a problemas que derivan en la asignación de personal, sub/sobreestimación de riesgos y problemas de calidad del producto final. Para minimizar este escenario, la estimación se apoya en técnicas y metodologías que han evolucionado con el tiempo, desde las primeras aproximaciones empíricas hasta modelos elaborados basados en datos históricos y teorías estadístico-probabilísticas.

PCU

Un método de estimación que ha cobrado importancia desde la expansión de la POO es la técnica basada en los Puntos de Caso de Uso. La idea es asumir que los casos de uso, como representaciones de interacciones entre las funcionalidades del sistema y sus actores, guardan relación con la complejidad y el volumen de trabajo en un proyecto de software. Para aplicar esta técnica, se identifican los casos de uso del sistema y se les asigna un peso según su complejidad (baja, media o alta). Se suman estas ponderaciones para obtener un valor total que refleja el tamaño del proyecto en términos de “puntos de caso de uso” (puntos de caso de uso sin ajustar). Además de la complejidad individual de cada caso de uso, también se consideran factores técnicos y ambientales que pueden influir en el esfuerzo, tales como la experiencia del equipo, la infraestructura o el nivel de requisitos de seguridad, entre otros. Una vez realizado este ajuste, se obtiene un valor final en puntos de caso de uso que se convierte en una base para la planificación (multiplicando los puntos por un factor de productividad), el presupuesto y la asignación de recursos.

Esta metodología se adopta con frecuencia en proyectos donde el modelado de casos de uso es utilizado en proceso de análisis y diseño.

Ventajas:

- Facilita la comunicación con todas las partes interesadas, dado que el mapa de casos de uso suele ser comprensible para quienes no son expertos en programación.
- Al conectar de forma directa la funcionalidad con la complejidad percibida, la técnica de Puntos de Caso de Uso resulta útil para anticipar cuán demandantes serán ciertas partes del sistema.

Desventajas:

- Su precisión depende en buena medida de la experiencia y criterio de quienes realizan la evaluación de la complejidad, así como de la calidad de los requisitos y la estabilidad de las especificaciones del sistema.

PF

Este método proviene de la necesidad de medir la funcionalidad entregada al usuario final, independientemente del lenguaje de programación o la plataforma empleada. Para tal fin, se contabilizan cinco tipos de componentes:

- Entradas externas (datos o transacciones que el usuario introduce al sistema),
- Salidas externas (información procesada que el sistema provee al usuario),
- Consultas externas (peticiones de datos en las que se realiza un intercambio con el sistema),
- Archivos lógicos internos (información que se mantiene y gestiona dentro del propio sistema)
- Interfaces externas (archivos o data stores compartidos con sistemas externos).

A cada uno de estos elementos se le asigna un valor según su complejidad: baja, media o alta. La suma ponderada de estos valores desemboca en un total de Puntos de Función que, con un factor de ajuste en caso de considerarse aspectos técnicos y de entorno, permite estimar el tamaño de la aplicación (similar a los PCU).

Los Puntos de Función disocian la lógica de negocio del stack de tecnologías y paradigmas de programación, ofreciendo una estimación que puede usarse para realizar comparaciones históricas entre proyectos de distinta naturaleza. En ámbitos corporativos, donde se manejan grandes volúmenes de datos y sistemas diversos, suele tener relevancia. La “exactitud” de la estimación con Puntos de Función suele crecer cuando hay datos históricos y cuando se conocen adecuadamente los tipos y frecuencias de cada operación que el usuario ejecuta sobre el sistema. Asimismo, su eficacia se incrementa si los equipos poseen experiencia en contar y categorizar cada componente según sus características y grado de complejidad.

Poker Planning

Con el surgimiento y auge de las metodologías ágiles, donde las iteraciones son cortas y la adaptación es más requerida, aparecieron métodos colaborativos y enfocados en la participación del equipo. Entre estos se encuentra Poker Planning, una variante de la técnica conocida como estimación por consenso. En esta dinámica, cada integrante del equipo recibe un conjunto de cartas con valores que suelen estar basados en la sucesión de Fibonacci (1, 2, 3, 5, 8, 13, 21, 34, etc.) o en escalas similares. Las historias de usuario a estimar se discuten brevemente y luego cada persona selecciona en secreto una carta que refleje su visión acerca del esfuerzo requerido. Todos revelan sus cartas al mismo tiempo, y luego se debaten las diferencias, sobre todo cuando alguien estima muy alto o muy bajo en comparación con el resto. A través de conversación y ahondando en detalles sobre las implicancias de cada tarea, se aclaran dudas y se exponen razones que pueden pasar desapercibidas si el proceso de estimación fuera individual. Finalmente, se repite la votación hasta que se alcanza un consenso o se determina un valor promedio razonable.

La principal fortaleza del Poker Planning radica en la riqueza de información que se comparte en cada ronda de estimación, además de fomentar la implicación de todos los miembros del equipo, desde programadores hasta analistas y, cuando corresponde, incluso representantes de negocio (PO, dueño de producto). De este modo, la estimación ya no recae en una sola persona o en el rol de un “experto” aislado. Más bien se convierte en un proceso colectivo que aporta diversos puntos de vista, revelando riesgos potenciales o dependencias no evidentes. La precisión final se incrementa gracias a la combinación de la experiencia de múltiples individuos y la transparencia de la discusión.

A su vez, el concepto de Story Points aparece muy ligado a la estimación ágil. Un Story Point es una medida abstracta que combina la complejidad y el esfuerzo de una historia de usuario, contemplando también los riesgos y el nivel de incertidumbre. Cuando el equipo asigna un número de Story Points a cada historia, lo hace con la intención de establecer una comparación relativa entre las distintas tareas del backlog. No se trata únicamente de pensar cuántas horas de trabajo conlleva, sino de determinar qué tan difícil es una tarea en comparación con otra. Por ejemplo, una historia que reciba 5 puntos se presume que es más compleja que otra de 2 puntos, aunque la relación no es necesariamente proporcional en términos de horas reales de trabajo. Más bien, estos puntos generan un mapa global del esfuerzo requerido para todo el conjunto de tareas, lo que ayuda a planificar la capacidad del equipo en cada iteración o sprint y a priorizar los elementos del backlog de manera coherente.

PERT / 3 puntos

Otro enfoque, denominado estimación de Tres Puntos o método PERT, se basa en la teoría estadística para afrontar la incertidumbre propia de la planificación de proyectos. El fundamento de PERT (Program Evaluation and Review Technique) consiste en disponer de tres estimaciones para cada tarea: una optimista, una pesimista y una más probable. Posteriormente, se calcula un valor medio ponderado mediante la fórmula $(O + 4M + P) / 6$, donde O es la estimación optimista, M la más probable y P la pesimista. Al tener en cuenta tanto la mejor como la peor situación posible y dar más peso a la estimación considerada más factible, se obtiene un promedio que representa una visión más global. Esta técnica se emplea con frecuencia en proyectos de gran envergadura y con alto grado de incertidumbre, puesto que las variaciones entre escenarios pueden ser muy marcadas y es valioso contemplar el riesgo de que las cosas no salgan según lo esperado. Además, PERT permite, junto con cálculos adicionales, determinar la desviación estándar de la estimación y, por tanto, establecer márgenes de contingencia o intervalos de confianza.

COCOMO

Existe una aproximación algorítmica representada históricamente por COCOMO (Constructive Cost Model), un modelo clásico propuesto por Barry Boehm. COCOMO vincula el tamaño del software, medido tradicionalmente en KLOC (miles de líneas de código), con el esfuerzo estimado (medido en persona-meses, generalmente), a través de una fórmula de la forma

$$\text{Esfuerzo} = a \times (\text{KLOC})^b \times F$$

Donde los valores de a y b dependen del tipo de proyecto (orgánico, semi-acoplado o integrado) y F es un factor de ajuste que recoge variables sobre el tipo de producto, el hardware, la experiencia del personal y otros factores del proyecto. Un proyecto orgánico se caracteriza por tener pocos requisitos cambiantes, un equipo pequeño y una solución relativamente simple, mientras que uno integrado o embedded puede requerir un alto nivel de innovación y un grado de complejidad técnica y organizativa muy superior.

COCOMO parece un enfoque algo antiguo (lo es en escala de tiempo), pero es una base conceptual que demuestra que, con datos históricos suficientes y ciertos supuestos, es posible crear fórmulas que capturen la relación entre tamaño, complejidad y esfuerzo. En la práctica, el modelo ha evolucionado a versiones más sofisticadas como COCOMO II, que incluye consideraciones acerca de la reutilización de código, el desarrollo ágil, la integración con métodos orientados a objetos y otros factores contemporáneos. La esencia siempre radica en que la productividad de un equipo no aumenta linealmente con el número de integrantes y que existen múltiples variables contextuales que condicionan el esfuerzo total requerido. Identificarlas y ponderarlas adecuadamente es parte fundamental de la estimación algorítmica.

A modo de conclusión...

Los métodos basados en Puntos de Caso de Uso y Puntos de Función se centran principalmente en la funcionalidad del sistema, mientras que Poker Planning y Story Points realzan la estimación cooperativa y la relativa simplicidad de no anclar la planificación a horas exactas. La estimación de Tres Puntos aborda directamente la incertidumbre probabilística, y COCOMO, por su parte, subraya la relación estadística entre líneas de código y esfuerzo, incluyendo factores de productividad y calidad del equipo. Si bien ninguna técnica es universalmente superior, es posible combinarlas o escoger la que mejor se adapte a las circunstancias particulares, como la experiencia del equipo, el grado de definición de requisitos y la cultura organizacional.

En el contexto de la medición de software, además de estos métodos de estimación, pueden identificarse modelos de medición que aportan a la comprensión del progreso, el rendimiento y la calidad del proyecto. Un modelo de medición establece un conjunto de métricas cuantitativas para analizar la evolución del desarrollo y facilitar la toma de decisiones. COCOMO es un claro ejemplo de modelo algorítmico, pero existen otras aproximaciones como las basadas en Puntos de Función y Puntos de Caso de Uso, que pueden integrarse a herramientas de seguimiento para medir no sólo cuánto se ha avanzado en términos de funcionalidad, sino también cómo evoluciona la productividad del equipo a lo largo del tiempo. En entornos ágiles, las métricas de velocidad, quemado (burn-down) y quemado ascendente (burn-up) ofrecen información sobre el ritmo de completitud de las historias de usuario y la tendencia de crecimiento o reducción del backlog. De esta manera, la estimación deja de ser un acontecimiento aislado al principio del proyecto y se convierte en un proceso continuo de medición y reajuste que acompaña el desarrollo iterativo.

Dentro de la administración de proyectos de software, la descomposición de tareas emerge como otra práctica primaria: desglosar el producto final en elementos cada vez más pequeños (y manejables) hasta alcanzar un nivel de detalle que facilite la estimación y la asignación de responsabilidades. Al dividir el trabajo en segmentos manejables, los equipos pueden asignar mejor a las personas adecuadas, abordar los riesgos de manera progresiva y realizar estimaciones más precisas para cada trozo de funcionalidad. Por ejemplo, una historia de usuario que inicialmente se describe a grandes rasgos, puede subdividirse en diversas subtareas: la creación de una interfaz, la lógica de negocio subyacente, la

manipulación de la base de datos y las pruebas de integración. Cada uno de estos pasos puede estimarse por separado, lo que contribuye a una mayor precisión al evitar la tentación de realizar promedios excesivamente generales. Algo a tener en cuenta cuando se aplica esta estrategia es considerar el esfuerzo necesario para volver a “ensamblar” las partes: la integración de las partes es un aspecto que no suele ser ponderado como corresponde en la práctica cotidiana.

Para que la descomposición de tareas sea efectiva, se recomienda un equilibrio entre el exceso de detalle y la falta de especificación. Si se fragmentan las tareas en partículas diminutas, el equipo de desarrollo puede perder una visión global del proyecto y la gestión se vuelve burocrática. Por el contrario, si se opta por tareas demasiado grandes, la estimación se torna poco confiable y la planificación se complica. Encontrar el punto óptimo depende de la experiencia del equipo y el contexto del proyecto. En metodologías como Scrum, se acostumbra que las tareas dentro de un sprint no sobrepasen determinados criterios, por ejemplo, que cada una pueda completarse en menos de un día o dos de trabajo; eso obedece a la filosofía de que si algo no puede descomponerse, seguramente oculta complejidades que dificultan la estimación y la gestión.

En relación con los conceptos clave para la estimación de proyectos de software, vale la pena profundizar en cuatro ideas esenciales: tamaño, esfuerzo, complejidad y duración. El tamaño se refiere a la magnitud o alcance del trabajo que se va a llevar a cabo. En métodos como Puntos de Función, implica calcular el número de entradas, salidas, archivos lógicos y demás componentes, mientras que en COCOMO se relaciona con el número de líneas de código. El esfuerzo, por su parte, denota la cantidad de recursos humanos invertidos en el proyecto, a menudo medidos en hombre-horas o hombre-meses. Es fundamental entender que un incremento en el tamaño del software no siempre se traduce linealmente en un incremento del esfuerzo, porque intervienen factores como la eficacia de las herramientas, la reutilización de código y el grado de automatización de pruebas.

La complejidad se encuentra asociada a cuán difícil resulta la tarea en cuestión. No es igual desarrollar una funcionalidad sencilla que involucre un único formulario y una base de datos pequeña, que emprender un sistema altamente distribuido con múltiples servicios, alta concurrencia y exigencias de seguridad. Las técnicas de estimación deben tener en cuenta la complejidad, ya sea de manera implícita (como hacen los Story Points al combinar esfuerzo, riesgo e incertidumbre) o explícita (como hacen los Puntos de Caso de Uso al clasificar los casos de uso en simples, medios o complejos junto con los factores de ajuste). La duración, en última instancia, es el lapso de calendario que va a requerir el proyecto. Este factor puede estar determinado por fechas impuestas externamente, como las metas comerciales o compromisos contractuales, o bien derivarse de la propia estimación del esfuerzo y la asignación de recursos. En algunos casos, la duración está sujeta a la Ley de Parkinson, pues si se asigna un tiempo excesivo, es probable que el equipo expanda el trabajo para ajustarse a dicho plazo, aunque en teoría podría haberse completado antes.

La interacción entre estos cuatro conceptos clave (tamaño, esfuerzo, complejidad y duración) es el eje central de la estimación. Toda metodología existente se va a fijar de una u otra forma en estos elementos para proponer una cifra que ayude a planificar. Es por esto que, en la práctica, la elección de una técnica suele basarse en factores como la disponibilidad de datos históricos, la naturaleza del proyecto, la cultura del equipo y el grado

de exactitud requerido. En proyectos muy pequeños y con baja complejidad, el equipo puede, en ocasiones, valerse de la simple experiencia o de la técnica ágil de Poker Planning para llegar a acuerdos rápidos. Si en cambio se trata de grandes proyectos corporativos con requisitos muy detallados, podría resultar más conveniente contar y evaluar cada funcionalidad a través de Puntos de Función o Puntos de Caso de Uso, siguiendo un proceso riguroso que brinde seguridad a los gerentes y patrocinadores.

No se puede pasar por alto que la estimación de proyectos de software no es una disciplina estática, y que resulta habitual reestimar a lo largo del proyecto. Esta práctica de reestimación o ajuste continuo surge porque, conforme se avanza, se adquiere mayor conocimiento sobre las dificultades técnicas, se clarifican requisitos y se completan tareas que sirven como referencia para las venideras (aunque es verdad que en proyectos con altos componentes legales-contractuales esto no es una práctica sencilla...). En metodologías ágiles, las reestimaciones se realizan incluso en cada sprint o iteración, pues se utilizan los datos de velocidad (la cantidad de Story Points completados en cada ciclo) para refinar lo que se puede hacer en el próximo. En entornos más predictivos o tradicionales, puede llevarse a cabo una reestimación en determinados hitos del proyecto cuando se ha completado cierta fase (por ejemplo, al finalizar el análisis o la primera etapa de diseño). De cualquier modo, es una buena idea asumir que la estimación inicial difícilmente se mantendrá inalterada hasta el final, ya que la propia naturaleza del desarrollo de software invita a la evolución de requisitos y al descubrimiento de otros nuevos.

Otro de los factores clave en la calidad de la estimación es la cooperación entre las distintas partes. Aunque existan modelos y ecuaciones complejas, existe cierto consenso en la industria sobre el aporte de los desarrolladores, analistas, testers y gerentes/líderes de proyecto es clave. Cada rol tiene una visión particular y maneja información que no siempre está contenida en la documentación del proyecto. Por ejemplo, un desarrollador experimentado puede detectar un problema técnico o una complejidad que no salta a la vista si se cuenta meramente el número de casos de uso o líneas de código. Un tester puede advertir que ciertas funcionalidades demandarán un extenso ciclo de pruebas y por lo tanto implicarán un mayor esfuerzo. La conjunción de los distintos puntos de vista enriquece la estimación y la hace más robusta ante imprevistos.

Es importante recalcar que estimar proyectos de software nunca implica obtener una certeza. Pero esto no quita que una estimación, con cualquier herramienta/método, es mejor que no tener una estimación. Por otro lado, también hay que destacar que una subestimación (tiempo o recursos insuficientes) es tan malo como una sobreestimación (tiempo o recursos de sobra): una estimación debe ser, siempre y en todo lugar, un ejercicio para obtener un resultado REALISTA.